



Relatório de Desenvolvimento
Simulador de Escalonamento de Processos
Round Robin com Feedback

Sistemas Operacionais

RAFAEL FÜHR MARONEZ
CARLOS EDUARDO SEIDL
LUCAS FREITAS JANZEN
EDUARDO LENHARDT FELTES

Baseado no repositório público [eduardlf/trabalho-round-robin](https://github.com/eduardlf/trabalho-round-robin)



1. Introdução

Este trabalho apresenta o desenvolvimento de um simulador de escalonamento de processos voltado para a disciplina de Sistemas Operacionais. O objetivo principal é reproduzir, de forma simplificada e observável por logs, o comportamento de uma CPU multitarefa, mostrando como processos competem por tempo de processamento, entram em espera, sofrem preempção e retornam após operações de entrada e saída.

2. Descrição do algoritmo

A solução implementa Round Robin com Feedback usando duas filas de pronto: uma fila de alta prioridade e outra de baixa prioridade. Todo processo novo entra inicialmente na fila de alta prioridade. Enquanto houver processos nessa fila, a CPU sempre escolhe um deles antes de consultar a fila de baixa.

A execução acontece em unidades discretas de tempo. A cada ciclo, o simulador verifica chegadas, atualiza retornos de I/O, incrementa o tempo de espera dos processos prontos, seleciona o processo seguinte quando a CPU está livre e executa uma unidade de CPU.

3. Premissas da simulação

O simulador foi configurado com parâmetros fixos para facilitar a demonstração e a reprodução dos resultados.

Premissa	Valor / comportamento
Quantum	3 unidades de tempo.
Quantidade de processos	10 processos gerados na simulação.
Chegada dos processos	Entre 0 e 15 unidades de tempo.
Tempo total de CPU	Entre 5 e 20 unidades de tempo.
Dispositivos de I/O	Disco, fita magnética e impressora.
Tempos de I/O	Disco = 3, fita = 5, impressora = 8 unidades.
Semente aleatória	Seed fixa 42 para execução determinística; opção de seed aleatória disponível.

O uso de seed fixa é útil para testes e correção, porque gera sempre o mesmo cenário. Quando a geração aleatória é habilitada, o simulador passa a criar cenários distintos a cada execução.

4. Estruturas de dados

A implementação segue uma abordagem centralizada em uma única classe principal, com uso de enums, records, listas e filas simples.

O PCB foi representado pelo record `Processo`, que guarda atributos como PID, PPID, tempo de chegada, tempo total de CPU, tempo restante, instante de I/O, dispositivo de I/O, prioridade atual, estado atual e métricas acumuladas de espera, finalização, execução de CPU e indicação de que o I/O já foi realizado.

Os eventos pendentes de entrada e saída foram representados pelo record `EventoIO`, contendo o PID, o dispositivo envolvido e o instante em que o processo deve retornar.

As filas de pronto foram modeladas com `Queue<Integer>`, armazenando apenas os PIDs dos processos. Já a lista principal de processos usa `List<Processo>`, permitindo localizar e atualizar cada PCB ao longo da simulação. A fila de I/O usa `List<EventoIO>` para permitir varredura e remoção segura dos eventos concluídos.

5. Fluxo de execução

O fluxo do simulador pode ser resumido nas etapas abaixo:

- Criação dos processos com atributos aleatórios dentro dos intervalos definidos.
- Inserção de cada processo novo na fila de alta prioridade quando sua hora de chegada é atingida.
- Verificação de retornos de I/O e reinserção do processo na fila correta conforme o dispositivo usado.
- Atualização do tempo de espera de todos os processos que estão prontos.
- Seleção da CPU: primeiro fila alta, depois fila baixa, sempre com novo quantum.
- Execução de uma unidade de CPU por ciclo, com testes imediatos de bloqueio, finalização ou preempção.
- Encerramento da simulação quando todos os processos atingem o estado FINALIZADO.

O bloqueio por I/O ocorre quando o processo atinge o instante configurado para solicitar o dispositivo. Nesse caso, ele sai da CPU, muda para BLOQUEADO e gera um evento com o momento exato de retorno.

A política de feedback dá preferência à fila alta. Quando o processo volta de disco, ele retorna à fila baixa; quando volta de fita ou impressora, ele retorna à fila alta. Isso cria um mecanismo de priorização que reflete a ideia de favorecer processos que já passaram por bloqueio de I/O.

6. Execução do projeto

O projeto foi documentado para execução em Java com suporte a Docker. A descrição do repositório indica uso de Java 17 com imagem Maven baseada em Eclipse Temurin, além de docker-compose para subir o ambiente.

Forma	Comando / explicação
Com Docker	<code>docker compose up -d --build</code> <code>docker compose exec app mvn -q clean compile exec:java -Dexec.mainClass=com.trabalho.br.round_robin.Round_robin</code>
Abrir o container	<code>docker compose up -d</code> <code>docker compose exec app bash</code>
Execução simplificada	O repositório contém o arquivo <code>executar.bat</code> para construir e executar o simulador diretamente no Docker.

O Dockerfile de exemplo descrito no README utiliza a imagem `maven:3.9.9-eclipse-temurin-17`, define `/app` como diretório de trabalho e mantém o container ativo com `tail -f /dev/null`. Na prática, isso facilita entrar no ambiente e rodar o Maven dentro do container sem precisar instalar dependências localmente.

7. Saída do simulador e interpretação

O programa registra eventos em formato de log com marcação de tempo, facilitando acompanhar o comportamento do escalonador.

```
[t=000] P1 chegou -> FILA ALTA  
[t=000] CPU selecionou P1 (ALTA)  
[t=001] Executando P1  
[t=002] Executando P1  
[t=003] Executando P1  
[t=003] Quantum expirou -> P1 para FILA BAIXA  
[t=004] P2 chegou -> FILA ALTA  
[t=004] CPU selecionou P2 (ALTA)
```

Nesse padrão de saída, o instante t indica o ciclo da simulação. A linha 'chegou -> FILA ALTA' mostra a ativação do processo. 'CPU selecionou' indica o despacho para a CPU. 'Executando' representa o avanço de uma unidade de tempo. Quando o quantum expira, o processo é rebaixado para a fila baixa. Em outras execuções, também podem aparecer mensagens de bloqueio por disco, fita ou impressora, seguidas de retorno à fila apropriada.

Ao final, o simulador imprime um relatório com tempo total, CPU ociosa, utilização da CPU, número de preempções, quantidade de eventos de I/O por dispositivo, tempo médio de espera, turnaround médio e o detalhamento individual de cada processo.



8. Conclusão

O trabalho evidencia, na prática, como políticas de escalonamento podem ser simuladas e observadas em tempo discreto. A estratégia Round Robin com Feedback mostrou-se adequada para demonstrar preempção, prioridade dinâmica e efeitos de I/O sobre a ordem de execução.

Entre os principais aprendizados estão a organização do estado de cada processo, o controle de filas e a atualização correta das métricas. A principal dificuldade tende a ser manter a coerência entre CPU, bloqueio e retorno de I/O sem perder rastreabilidade.

Como melhorias futuras, seria interessante separar a lógica em classes menores, adicionar visualização gráfica da simulação, incluir mais métricas comparativas e permitir a configuração dos parâmetros por arquivo externo ou interface.